

SYMBOLIC MICROPROGRAM:-

Control storage ^{words} 128 $\rightarrow$ 20 Bit words.

The first 64 words : Routines for 16 machine inst.

The last 64 words: Used for other purposes ✓
(e.g. fetch routine & other subroutines)

Mapping: Opcode XXXX into OXXXXX00, the first address for 16 routines are 0(00000), 4, 8, 12, 16, 20 ... 60.

ADDITION & INDIRECT BRANCHING:-

LABELS.	MICROPS	CD	BR	AD
	ORG 0			
ADD:	NOP	I	CALL	INDRCT
	READ	U	JUMP	NEXT
	ADD	U	JMP	FETCH.

To see how the transfer and return from the indirect sub routine occurs

INDRCT: READ	U	JMP	NEXT
	DRTAR	U	RET

$\rightarrow$ (a)

assume that MAP at the end of the fetch inst caused a branch to address 0 from where the ADD(sub)routine started.

The first micro-inst in ADD calls the subroutine INDRCT conditioned at I. If $I=1$, a branch to INDRCT occurs and the return address is stored in the subroutine register (SBR). The INDRCT has 2 microinst as described in (a).

* INDIRECT address considers the address part of the inst as the address where the effective address is stored rather than the address of the operand.

$\therefore$ the memory has to be accessed to get the effective address that is then transferred to AR.

The return from the subroutine RET transfers the address from SBR to CAR, thus we return to the second micro inst. of ADD routine.

The 2nd micro inst of ADD; reads operand from memory into the DR

The 3rd " " " : adds the contents of DR and AC; and then JUMPs back to the beginning of the fetch routine.

BRANCH ROUTINE:-

the eff. add. is then transferred from AR to PC and micro prog jump back to fetch routine.

BRANCH:	ORG 4	S	JMP	OVER	the micro inst here may/does call an indirect subroutine if I = 1
	NOP	U	JMP	FETCH	
OVER:	NOP	I	CALL	INDIRECT	
	ART PC	U	JMP	FETCH	

The Branch instruction should cause the branch to the effective address if $AC < 0$. The AC is less than 0 if sign bit is negative which is detected from S being equal to 1 from the status bit.

If $S = 0$; no branch would occur and the next micro inst would cause a jump back to fetch routine without altering the contents of PC.

STORE ROUTINE:-

	ORG 8			
STORE:	NOP	I	CALL	INDRCT
	ACTDR	U	JMP	NEXT
	WRITE	U	JMP	FETCH.

EXCHANGE ROUTINE:-

	ORG 12.			
EXCHANGE:	NOP	I	CALL	INDRCT
	READ	U	JMP	NEXT
	ACTOR, DRTAC	U	JMP	NEXT
	WRITE	U	JMP	FETCH.

BINARY MICROPROGRAM:-

The symbolic microprogram must be translated into Binary either by the assembler or the user if the micro-program is simple enough.

Let's consider the example of ADD.

ADD routine has only 3 microinstructions 0, 1 & 2.

The next routine starts at address 4. Note that address 3 has no Binary equivalent in symbolic microprogram.

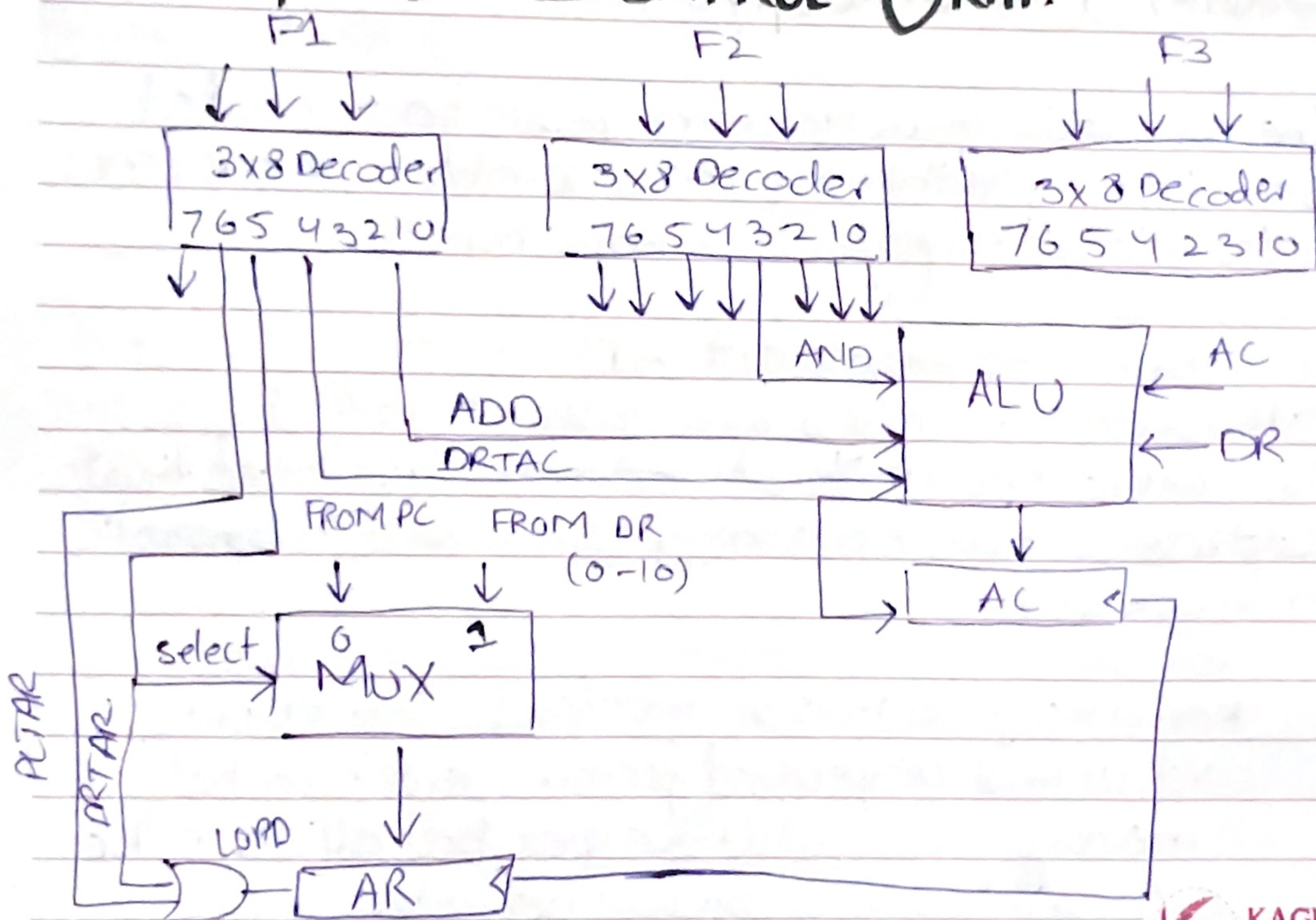
* even though address 3 is not used, some binary value must be specified for each word in control memory. We could have specified all 0s in the word since this location will never be used.

- * However, if some unforeseen error occurs, or if a noise signal sets CAR to the value 3, it will be wise to jump address 64, which is the beginning of the fetch routine.

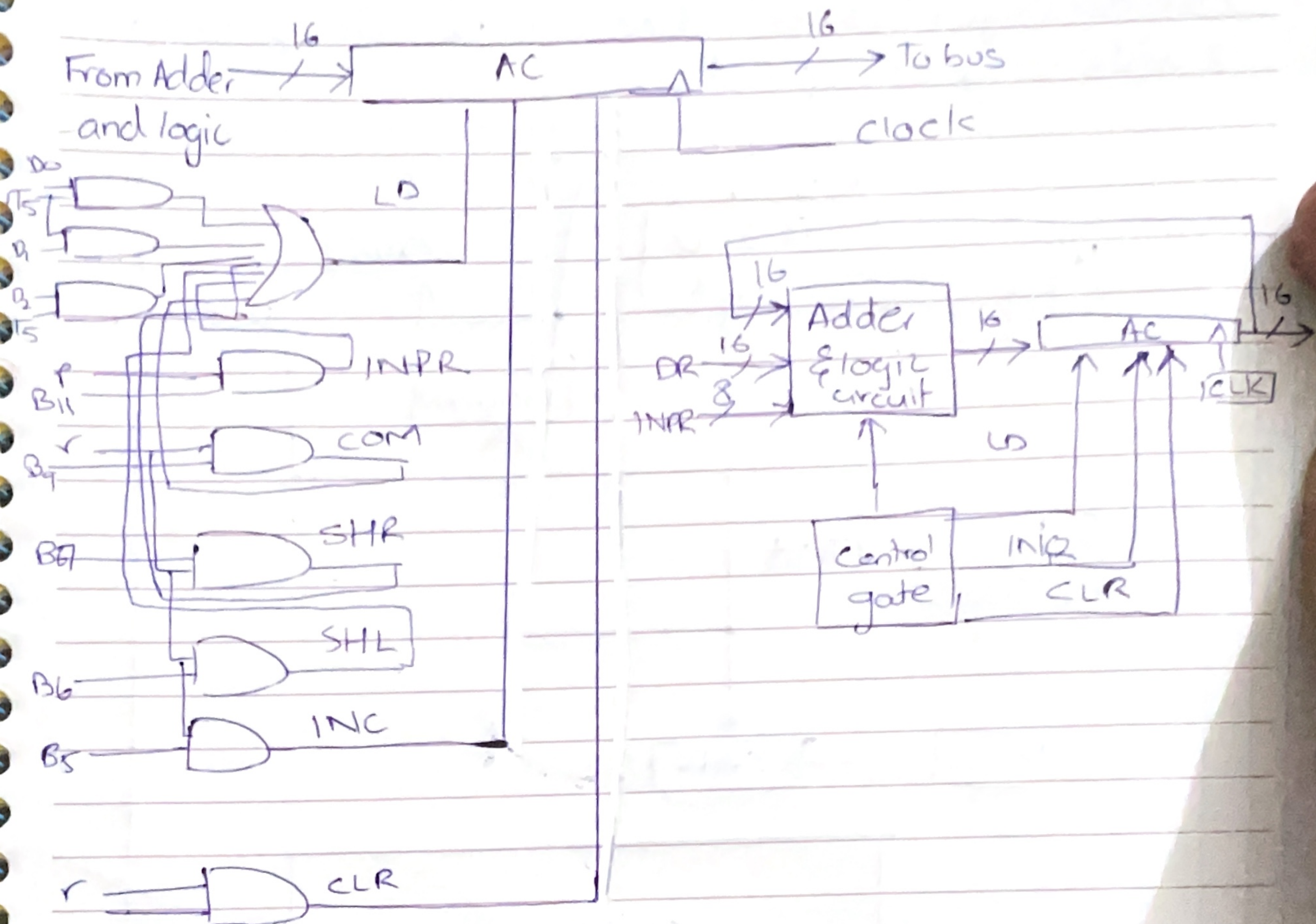
Micro-Routine	Address	Binary Micro-Instruction
ADD	Decimal	Binary
		F1 F2 F3 CD BR AD
0	0000000	000 000 000 01 01 1000 011
1	0000001	000 100 000 00 00 00000 10
2	0000010	001 000 000 00 00 1000000
3	0000011	000 000 000 00 00 1000000

- * Check reprinting from Book A pg no. 240. Imp!

DESIGN OF CONTROL UNIT.



Arithmetic logic design from chapter no. 5



Read Description from pg no. 242 if need!

MICRO-PROGRAM SEQUENCER:-

The basic components of a microprogrammed control unit are the control memory and the circuits that select the next address.

→ The address selection part is called the microprogram sequencer.

→ A microprogram sequencer can be constructed using digital functions to suit a particular application.

To guarantee a wide range of acceptability, an integrated circuit sequencer must provide an internal organization that can be adapted to a wide range of applications.

